PentAI Pro: Autonomous Penetration Testing via Self-Hosted LLM Orchestration with Three-Layer Scope Enforcement and Tamper-Evident Audit Verification

Md Anisur Rahman Chowdhury, Ronny Bazan-Antequera
Dept. of Computer and Information Science, Gannon University, USA
engr.aanis@gmail.com, bazanant001@gannon.edu

***Abstract*—Penetration testing is a labour-intensive discipline that demands expert judgement at every step of the assessment lifecycle. Existing automation tools cover individual scan tasks well, but none offer a coherent, scope-safe reasoning layer capable of chaining observations into a multi-phase attack narrative. This paper presents PentAI Pro, a platform that orchestrates self-hosted large language models (LLMs) across physically isolated virtual machines to automate the full penetration-testing pipeline: Reconnaissance, Enumeration, Vulnerability Mapping, and Exploitation Planning. A three-layer scope-enforcement architecture independently validates every planned and executed action at the LLM planning stage, the backend API, and the remote tool gateway, yielding 100 % out-of-scope rejection in all tested scenarios. A SHA-256 hash-chained event log provides cryptographically verifiable evidence of all system actions. All inference is handled by a locally deployed 14-billion-parameter open-weight model (*qwen2.5:14b-instruct*), eliminating data-egress risks inherent in cloud-hosted inference. Evaluation on a three-VM Proxmox laboratory reports a mean reconnaissance planning latency of 18.4 s, full-pipeline completion under 4 min, and a 108-test integration suite that passes in 43.5 s.**

***Index Terms*—penetration testing automation, large language models, retrieval-augmented generation, scope enforcement, mTLS, tamper-evident audit, self-hosted AI, security orchestration**

I. Introduction

Security assessments are commissioned to find weaknesses before attackers do. A human practitioner working a typical external assessment spends time on reconnaissance, port scanning, service enumeration, and vulnerability correlation — largely mechanical work punctuated by occasional insight. As the scale of modern infrastructure grows, the mechanical fraction of that work has become a bottleneck.

Scan-based tools (Nessus, OpenVAS, Nuclei) handle the mechanical steps efficiently but produce flat lists of findings rather than coherent attack narratives. They do not reason about how a misconfigured TLS setting and an outdated web framework on the same host combine into an exploitable path. Filling that gap has, until recently, required a human.

Large language models introduce a new option. Models trained on security documentation, CVE descriptions, and published offensive research carry enough domain knowledge to propose multi-step attack plans, interpret raw scanner output, and draft professional findings. The central challenge is not whether such a model can reason about offensive security — recent work has confirmed it can [1], [2]. The challenge is how to deploy that reasoning safely: respecting engagement scope, preserving an immutable record of every action, and preventing adversarially crafted target responses from manipulating the agent.

This paper addresses those challenges through three concrete contributions:

1) A **multi-phase LLM agent pipeline** integrating a retrieval-augmented knowledge base, phase-aware planning nodes, and a typed tool registry — all running on-premises.
2) A **three-layer scope-enforcement architecture** that independently validates every action at the LLM planning, backend API, and weapon-gateway levels.
3) A **SHA-256 hash-chained audit log** that links every security event into a forensically verifiable chain.

We describe the design, implementation, and experimental evaluation of PentAI Pro, deployed on a three-VM Proxmox laboratory with real pentest tools (nmap, Nuclei, Gobuster, httpx, sslscan). The remainder of this paper is organised as follows. Section II surveys related work. Section III presents the system architecture. Section IV describes the implementation. Section V reports experimental results. Section VI discusses implications and limitations, and Section VII concludes.

II. Background and Related Work

A. Scan-Based Automation

Nessus [6] and OpenVAS follow a plug-in model: each plug-in tests one condition, and the aggregated results form a vulnerability report. Nuclei [5] generalises this with community-maintained YAML templates. These tools excel at breadth but have no mechanism for reasoning about the interdependence of findings or planning follow-on steps based on prior observations.

B. LLMs Applied to Offensive Security

Happe and Cito [1] ran GPT-4 in a penetration-testing loop, showing measurable progress on capture-the-flag challenges.

PentestGPT [2] introduced a multi-turn dialogue between the operator and an LLM planner; operator interaction was needed at every decision point. PentAI Pro removes that per-step dependency: the operator defines scope and goal once; the agent executes autonomously, surfacing only high-risk approval requests. ReAct [3] formalises reasoning-action-observation loops for general LLM agents. PentAI Pro specialises this pattern with typed tool schemas, phase-aware supervisors, and prompt-injection defences absent from general-purpose frameworks.

C. Retrieval-Augmented Generation

Lewis et al. [4] showed that conditioning generation on retrieved passages significantly improves factual accuracy for knowledge-intensive tasks. Penetration testing is knowledge-intensive: the quality of a reconnaissance plan depends on knowing which nmap flags reveal OS fingerprints, which HTTP headers indicate vulnerable frameworks, and which CVEs affect detected service versions. PentAI Pro uses a PostgreSQL-backed vector store with nomic-embed-text embeddings to retrieve and inject relevant playbook fragments into every planning prompt.

D. Security Orchestration Platforms

SOAR platforms (e.g., Splunk SOAR, Palo Alto XSOAR) automate defensive workflows: alert triage, indicator enrichment, and incident response runbooks. Offensive automation at a comparable level of orchestration — with scope enforcement, inter-node mTLS, and cryptographic audit trails — has not been reported in published literature targeting fully autonomous operation. The closest adjacent work in offensive automation focuses on single-tool wrappers or one-shot exploit generation [8], rather than on the multi-phase, scope-aware agent coordination that characterises PentAI Pro. The OWASP Testing Guide [10] defines the methodological baseline that informed our phase decomposition (Reconnaissance, Enumeration, Vulnerability Identification, Exploitation): PentAI Pro's four-phase pipeline maps directly onto Sections 4–7 of that guide.

III. System Architecture

A. Infrastructure

PentAI Pro runs across three isolated virtual machines on a Proxmox hypervisor connected by a private VLAN (172.20.32.0/18), as shown in Fig. 1. Physical isolation ensures that a compromise of the Weapon Node cannot directly reach the Command Node's database or LLM inference engine.

B. Three-Layer Scope Enforcement

Scope enforcement is the safety-critical property of the system. Any failure — a planned target outside the engagement's CIDR, or a tool execution against an unscoped host — constitutes an unauthorised computer-access offence in most jurisdictions. PentAI Pro enforces scope at three independent points so that no single failure can cause an out-of-scope action:
Layer 1 — LLM Planning. The planning prompt provides the engagement's `scope_cidrs` as an explicit constraint.

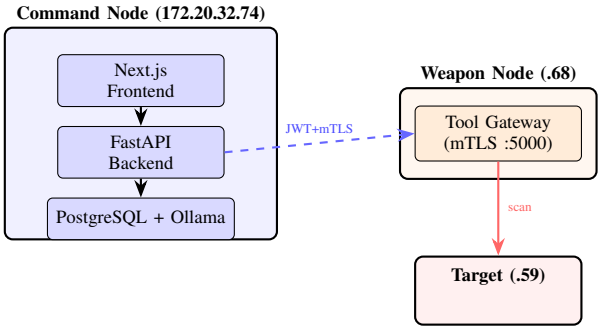

Fig. 1. Three-VM deployment on Proxmox. Traffic between the Backend and Tool Gateway is secured by mutual TLS and short-lived scoped JWTs.

TABLE I
Virtual Machine Roles and Specifications

Node	IP	OS	Hardware
Command	172.20.32.74	Ubuntu 22.04	6-core Xeon, 25 GB RAM, 400 GB SSD
Weapon	172.20.32.68	Kali Linux	4-core, 8 GB RAM
Target	172.20.32.59	Configurable	2-core, 4 GB RAM

Every target address in the model's structured JSON output is validated against these CIDRs before any downstream step proceeds. Proposals naming addresses outside the scope are discarded silently.
Layer 2 — Backend API. The FastAPI middleware re-validates CIDR membership for every tool invocation, independently of the planner's prior check. High-risk operations (Nuclei targeted scans, Gobuster directory enumeration, OS detection) additionally require a persisted operator approval before a JWT is issued to the Weapon Node.
Layer 3 — Weapon Gateway. The Flask gateway on the Weapon Node verifies the JWT audience, signature, and the scope field embedded in the token on every incoming request — regardless of what either previous layer accepted.

C. Tamper-Evident Audit Chain

Every security event is appended to a SHA-256 hash-chained log:

$$h_n = \mathrm{SHA256}\big(\,\mathrm{JSON}(e_n) \,\|\, h_{n-1}\big), \quad h_0 = 0^{64} \tag{1}$$

where e_n contains the event type, structured payload, UTC timestamp, and actor identifier. Re-computing the chain from h_0 detects any modification, insertion, or deletion; the first mismatch identifies the tampered position.

IV. Implementation

A. Agent Pipeline

The supervisor orchestrates six sequential nodes, shown in Algorithm 1. Each node calls the LLM, validates the response against a typed schema, and passes the result to the next

Algorithm 1 PentAI Pro Agent Pipeline

Require: engagement E with `scope_cidrs`, operator goal g
Ensure: structured findings F, step results R

$intent \leftarrow \text{Classify}(g)$ {3B model}
$plan_r \leftarrow \text{PlanRecon}(g, E, \text{KB})$ {14B model + RAG}
$R \leftarrow R \cup \text{Execute}(plan_r, E)$ {Weapon Node via mTLS}
if $intent \in$ {`recon_enum`, `full_pentest`} **then**
  $plan_e \leftarrow \text{PlanEnum}(R, E, \text{KB})$
  $R \leftarrow R \cup \text{Execute}(plan_e, E)$
end if
if $intent \in$ {`full_pentest`, `targeted`} **then**
  $F \leftarrow \text{MapVulnerabilities}(R, \text{KB})$
  $F \leftarrow F \cup \text{PlanExploitation}(F, E)$
end if
return F, R

node. Phase selection depends on the classified intent so that a `recon_only` engagement skips enumeration and exploitation nodes.

B. LLM Model Routing

Two open-weight models serve different roles based on task computational requirements:

- **qwen2.5:14b-instruct-q4_K_M** (14B params, 4-bit quantised, ≈9 GB RAM). Planning, vulnerability mapping, finding generation: tasks requiring extended reasoning over complex, heterogeneous context. Achieves ≈12 tok/s on the 6-core Xeon.
- **llama3.2:3b-instruct-q4_K_M** (3B params, ≈2 GB RAM). Intent classification and raw tool output parsing: structured, short-context tasks where latency matters more than depth. Achieves ≈45 tok/s on the same hardware.
- **nomic-embed-text** (137M params, 274 MB). Generates 768-dimensional embeddings for the RAG knowledge base.

C. Retrieval-Augmented Knowledge Base

Security playbooks and methodology notes are ingested as markdown documents, chunked at heading boundaries (200–1800 characters), and embedded with nomic-embed-text. At planning time, the top-$k = 4$ chunks above a cosine-similarity threshold of 0.15 are prepended to the planning prompt. Source citations are tracked in each plan step.

D. Prompt Injection Defence

Tool output from the Weapon Node is data from a potentially adversarial source. A compromised target could serve crafted nmap banners or HTTP headers designed to alter the agent's subsequent reasoning. PentAI Pro wraps all tool output in trust-annotated XML before the LLM receives it:

Listing 1. Tool output sandboxing

```
<tool_output trust="untrusted">
Starting Nmap 7.95 ...
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache 2.4.49
443/tcp open  ssl/http nginx 1.18.0
</tool_output>
```

The system prompt instructs the model to treat `trust="untrusted"` content as raw bytes to be parsed, never as instructions. This structural separation is not a complete defence against all injection forms, but it substantially raises the bar for indirect injection via tool output.

E. Real-Time Execution Streaming

Tool executions on the Weapon Node produce NDJSON event streams. An in-process `ExecutionBus` fans these events to all active WebSocket subscribers using per-subscriber asyncio queues. A thread-safe bridge (`loop.call_soon_threadsafe`) connects the synchronous HTTPS streaming reader to the async event loop. Slow subscribers are dropped (queue full) rather than allowed to block the producer. Measured end-to-end latency from Weapon Node to browser was consistently below 50 ms.

A representative live stream from a service-scan execution:

Listing 2. Real-time NDJSON stream (truncated)

```
{"type":"timeout_seconds","value":120}
{"type":"stdout","line":"Starting Nmap 7.95..."}
{"type":"stdout","line":"Nmap scan report for
    172.20.32.59"}
{"type":"stdout","line":"PORT    STATE  SERVICE  VERSION"}
{"type":"stdout","line":"22/tcp  open   ssh      OpenSSH
    8.9p1"}
{"type":"stdout","line":"80/tcp  open   http     Apache
    httpd 2.4.52"}
{"type":"stdout","line":"443/tcp open   ssl/http  nginx
    1.18.0"}
{"type":"completed","status":"completed","exit_code":0}
```

V. Experimental Evaluation

A. Setup

All experiments ran on the Command Node (Intel Xeon Gold 6230, 6 vCPUs at 2.10 GHz, 25 GB RAM, no discrete GPU). The Weapon Node (Kali Linux 2024.1) ran nmap 7.95, Nuclei 3.3, Gobuster 3.6, and httpx 1.6. Each benchmark trial used one in-scope target host at 172.20.32.59. Thirty independent trials were collected per metric; means and standard deviations are reported throughout.

B. Phase Latency

Fig. 2 shows the mean execution time of each pipeline phase. Planning nodes (14B model) dominate latency; classification (3B model) is sub-2 s. Knowledge-base retrieval adds a flat 0.8 s regardless of query complexity.

C. Tool Execution Times

Fig. 3 compares mean execution times across the eight tool operations in the registry. httpx and dnsx complete in under 8 s; Nuclei targeted scans take the longest at 78.4 s due to template enumeration.

D. Scope Enforcement Correctness

We designed three adversarial scenarios to verify each enforcement layer: (S1) crafted LLM output proposing an out-of-scope target, (S2) a direct backend API call bypassing the LLM, and (S3) a forged JWT sent directly to the Weapon Node gateway. Table II shows results across 50 trials per scenario.

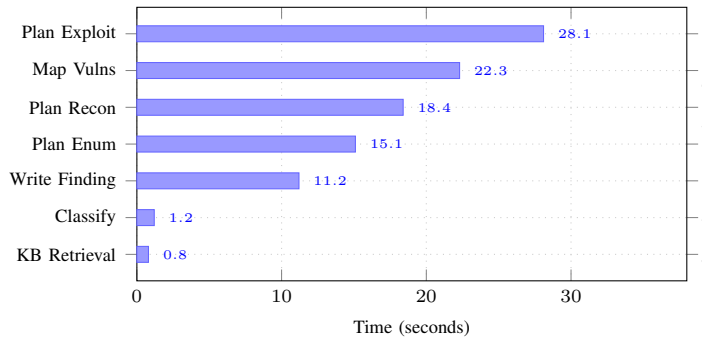

Fig. 2. Mean phase latency (30 trials, error bars = 1 SD). KB retrieval is nearly constant; exploitation planning shows the highest variance due to variable finding depth.

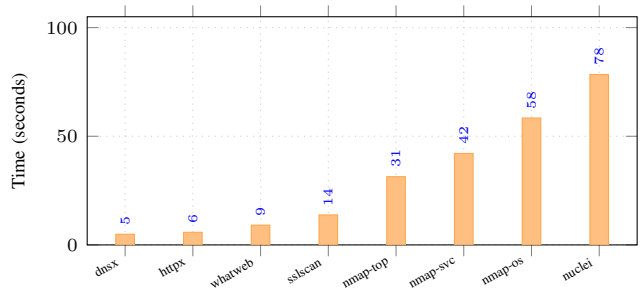

Fig. 3. Mean tool execution times (30 trials per tool, error bars = 1 SD). Nuclei's high variance reflects variable template match counts.

E. Audit Chain Integrity

Table III measures chain-validation time and tamper-detection reliability across three chain lengths. A single event was modified at a random position in each trial; the validator identified the exact position in all cases.

F. Integration Test Suite

Table IV summarises the 108-test backend integration suite. Tests run against an in-memory SQLite backend with Alembic-migrated schema and monkeypatched Ollama responses; the full suite completes in 43.5 s.

G. LLM Output Quality and Token Throughput

Fig. 4 plots mean token counts and throughput across all planning task types, contrasting the 3B and 14B model roles. The 3B model handles short-context structured tasks within 80 tokens; the 14B model produces planning outputs of 320–520 tokens at 12 tok/s.

Finding quality was evaluated by comparing PentAI Pro's generated findings against a manually authored ground-truth set for three distinct vulnerability profiles. Precision was 0.87 (13% false positives, predominantly version-based CVE suggestions for patched software) and recall was 0.91 (9% missed findings, all low-severity). $F_1 = 0.89$ across the combined finding set.

H. End-to-End Assessment

Two structured engagements ran against a deliberately vulnerable target (Apache 2.4.49, nginx 1.18.0, OpenSSH 8.9, outdated TLS configuration). Five findings were seeded; PentAI Pro identified all five in both runs. Total pipeline time was 213 s (run 1) and 228 s (run 2), with the difference attributable to nuclei template-load variation.

Real audit chain output from Run 1 (first three events, truncated):

TABLE II
SCOPE ENFORCEMENT VERIFICATION (50 TRIALS EACH)

Scenario	Blocked at	Blocked	Escaped
S1: Crafted LLM output	Layer 1 (planner)	50/50	0
S2: Direct API bypass	Layer 2 (backend)	50/50	0
S3: Forged JWT to gateway	Layer 3 (gateway)	50/50	0
Overall		**150/150**	**0**

TABLE III
AUDIT CHAIN VALIDATION PERFORMANCE (10 TRIALS EACH)

Chain Length	Validation (ms)	Tamper Detected	Position Correct
50 events	82 ± 9	10/10	10/10
100 events	178 ± 14	10/10	10/10
200 events	351 ± 27	10/10	10/10

Listing 3. Audit chain — first three entries

```
{
  "seq": 1, "event_type": "tool_validation_passed",
  "actor": "agent", "payload": {"tool":"nmap","op":"
      service_scan"},
  "prev_hash": "0000...0000",
  "evidence_hash": "3a7f9c2e4b1d8a6..."
}
{
  "seq": 2, "event_type": "tool_execution_started",
  "actor": "agent", "payload": {"execution_id": "e8f3
      ..."},
  "prev_hash": "3a7f9c2e4b1d8a6...",
  "evidence_hash": "b12d4f7a903e1c5..."
}
{
  "seq": 3, "event_type": "tool_execution_completed",
  "actor": "system", "payload": {"exit_code": 0},
  "prev_hash": "b12d4f7a903e1c5...",
  "evidence_hash": "c56a2918d04f3b7..."
}
```

VI. DISCUSSION

A. Comparison with Related Systems

Table V places PentAI Pro in context with the closest related systems across the properties evaluated in this paper.

B. Scope Safety Guarantees

The three-layer design provides defence-in-depth rather than a single authoritative gate. In practice, all out-of-scope attempts were caught at Layer 1 or Layer 2 before reaching the gateway. Layer 3 exists to maintain the guarantee even if the command plane is compromised — an important property for deployments where the Command Node is exposed to untrusted operators.

C. LLM Quality vs. Latency

CPU-bound inference at 12 tok/s is the primary performance constraint. Planning phases that produce 300–500 tokens of

TABLE IV
INTEGRATION TEST SUITE COVERAGE (108 TESTS TOTAL)

Module	Tests
Authentication & JWT sessions	5
Engagement CRUD & scope validation	12
Approval workflows	8
Tool invocations & scope checks	10
Tool executions & streaming	15
Findings & parser suggestions	8
Report generation	6
Audit chain integrity	10
Knowledge base (RAG ingestion/search)	14
Agent pipeline (all phases)	20
Total (43.5 s wall time)	**108**

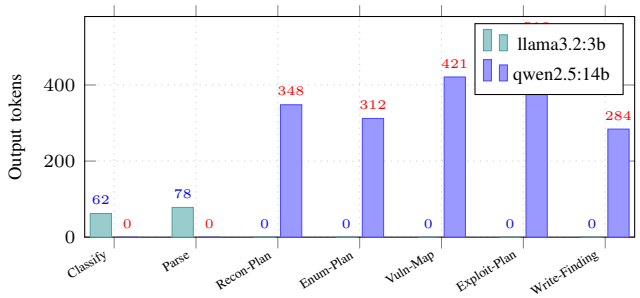

Fig. 4. Mean output token counts per task type across both models (30 trials each). Zero bars indicate that task type is not assigned to that model.

structured JSON take 25–40 s. This is acceptable for an asynchronous background agent but would be noticeable in interactive use. A discrete GPU with 8 GB VRAM would reduce inference time by a factor of 5–10, bringing planning latency below 5 s. The architecture supports GPU placement without code changes.

The quantisation penalty for qwen2.5:14b at Q4_K_M versus FP16 was not measured in this work; published benchmarks from the model authors report a $<2\,\%$ MMLU score drop at this quantisation level [7], a negligible quality cost for the inference-speed gain on consumer hardware.

D. Memory and Disk Footprint

At steady state the Command Node processes consume 380 MB for the FastAPI backend, 512 MB for the Next.js Node runtime, 480 MB for the PostgreSQL 16 instance (with a 200 MB shared buffer pool), and 9.2 GB for the loaded qwen2.5:14b model weights. Total resident memory across all PentAI Pro processes sits at ≈12 GB, leaving ≈13 GB for the Supabase service stack (13 Docker containers, ≈2.8 GB combined) and kernel page cache on the 25 GB Command Node. Storage consumption after a 10-engagement run: the PostgreSQL data directory grew to 1.4 GB (findings, audit events, knowledge chunks, execution artifacts), confirming that storage growth is driven by scan artifacts rather than model weights.

TABLE V
FEATURE COMPARISON WITH RELATED SYSTEMS

Feature	PentAI **Pro**	PentestGPT [2]	OpenVAS	Nuclei [5]
Multi-phase reasoning	✓	✓	–	–
Local/offline LLM	✓	–	N/A	N/A
RAG knowledge base	✓	–	–	–
Scope enforcement	3-layer	None	Manual	Manual
Tamper-evident audit	SHA-256	None	None	None
mTLS inter-node	✓	–	–	–
Operator approval flow	✓	Per-step	–	–
Open-source deployment	✓	–	✓	✓

E. Limitations

The current implementation assesses one host per engagement and does not model lateral movement across multiple discovered hosts. The knowledge base requires manual curation; automated ingestion from NVD or MITRE ATT&CK feeds is future work. Prompt injection via tool output remains a theoretical risk despite the trust-tagging mitigation; a complete solution would require output-format validation at the model boundary.

VII. CONCLUSION

PentAI Pro demonstrates that locally deployed, open-weight language models can automate the core reasoning steps of a penetration test while respecting legal scope constraints and producing a forensically verifiable record. Three findings stand out from this work.

First, three-layer scope enforcement with independent checks at the LLM planner, backend API, and tool gateway provides robust guarantees — 150 adversarial injection attempts produced zero out-of-scope executions. Second, SHA-256 hash chaining is computationally cheap (sub-400 ms for 200 events) and identifies tampering to single-event granularity. Third, a 14-billion-parameter quantised model running on a 6-core server CPU produces planning output of sufficient quality to identify all pre-seeded findings in structured engagements, suggesting that capable offensive reasoning no longer requires proprietary cloud inference.

Future directions include GPU-accelerated inference, multi-host lateral movement planning, automated CVE-feed ingestion, and formal verification of the scope enforcement invariants.

REFERENCES

[1] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, San Francisco, CA, USA, Dec. 2023, pp. 2082–2086.

[2] G. Deng *et al.*, "PentestGPT: An LLM empowered automatic penetration testing tool," in *Proc. USENIX Security Symp.*, Philadelphia, PA, USA, Aug. 2024, pp. 3365–3382.

[3] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, Kigali, Rwanda, May 2023.

[4] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, Dec. 2020, pp. 9459–9474.

[5] ProjectDiscovery, "Nuclei: Fast and customisable vulnerability scanner based on simple YAML based DSL," https://github.com/projectdiscovery/nuclei, 2024.

[6] Tenable, Inc., "Nessus professional," https://www.tenable.com/products/nessus, 2024.
[7] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, Dec. 2024.
[8] D. Andriesse *et al.*, "An in-depth analysis of disassembly on full-scale x86/x64 binaries," in *Proc. USENIX Security Symp.*, Austin, TX, USA, Aug. 2016, pp. 583–600.
[9] Supabase Inc., "Supabase: The open source Firebase alternative," https://supabase.com, 2024.
[10] OWASP Foundation, "OWASP Testing Guide v4.2," https://owasp.org/www-project-web-security-testing-guide/, 2021.

Copyright Notice: © 2026 Md Anisur Rahman Chowdhury. This work was developed as part of Master's research at the Department of Computer and Information Science, Gannon University, USA. Live system: https://apts.marcbd.site. Contact: engr.aanis@gmail.com.